

CAPSTONE 5

SahayakMap

The Flood That Nobody Maps, Nobody Predicts, and Everybody Drowns In

100xEngineers · Applied AI Capstone Series

The Situation

Rajesh Sharma is 44 years old, a battalion commander in India's National Disaster Response Force, stationed in Cuttack, Odisha. Every monsoon, he coordinates rescue operations across 15 or more districts. His current workflow: WhatsApp messages from local police stations with GPS pins that are often wrong, news channels showing footage that is six hours old, and Excel sheets tracking which boats are deployed where. When Cyclone Fani made landfall, the gap between "where flooding is happening now" and "where our boats actually are" cost his team 14 hours of response time. People died in that gap.

Rajesh does not lack data. India's Central Water Commission publishes river gauge levels. The India Meteorological Department broadcasts cyclone tracks and rainfall radar. ISRO's Bhuvan platform provides satellite imagery. Social media is full of geotagged posts from citizens standing in floodwater. ADS-B feeds show helicopter positions. OpenStreetMap has the road and bridge network. The data is free. It is public. It is real-time. And none of it talks to anything else.

Rajesh does not know that the Mahanadi at Naraj is 1.2 metres above danger level and rising at 0.3 metres per hour. He does not know that Twitter has 47 geotagged posts from Jajpur district showing flooded roads that the CWC gauges have not caught yet. He does not know that two of his five rescue boats are positioned in a district where the flood has already receded, while the district 80 kilometres east has no boats and water is rising. He does not know that the bridge on NH-16 near Balasore has been reported submerged by three independent sources in the last 45 minutes, which means his supply convoy is about to hit a dead end.

The Central Water Commission knows its gauge readings. IMD knows its radar. The district collector knows his ground reports. Twitter users know what they are standing in. Nobody knows all of it simultaneously. And the person who needs all of it simultaneously, the person deciding where to send 200 rescuers and 30 boats across 50,000 square kilometres of evolving floodplain, is working off WhatsApp forwards and television news.

This is not a hypothetical. India experienced over 300 flood events between 2019 and 2025. Floods and cyclones caused over 9,000 deaths and displaced over 50 million people in the last five years. The economic damage exceeds ₹2 lakh crore annually. NDRF deployed over 350 rescue teams during the 2024 monsoon season alone. Each team operated with incomplete, contradictory, and delayed information. The gap between available data and operational intelligence is not a technology problem. It is an integration problem. The data exists. The intelligence does not.

The information asymmetry here is structural. The data producers (CWC, IMD, ISRO, social media users, municipal CCTV operators) each see one dimension of the disaster. The data consumer (the field commander) needs all dimensions fused into a single, temporally coherent, spatially accurate operational picture. No system in India does this. The coordination happens over phone calls and group chats. The “situational awareness” is whatever the most recent message in the WhatsApp group says.

The crisis is not that India lacks disaster data. It is that nobody has built the intelligence layer that turns data into decisions.

What Exists Today

Rajesh’s current options:

Option A: Check each source manually. Open the CWC portal in one browser tab. Open IMD’s mausam.imd.gov.in in another. Pull up Bhuvan for satellite imagery. Scroll through the WhatsApp group. Call the district collector. Call the state emergency operations centre. By the time Rajesh has assembled a partial picture from five sources, 90 minutes have passed and the flood front has moved 12 kilometres. The information he assembled is already stale. This is what most commanders do. They operate on a mental model built from fragments, and the model is always behind the reality.

Option B: Use an existing GIS platform. Platforms like ESRI ArcGIS or QGIS can display geospatial layers. But they require someone to manually ingest, clean, and overlay each data source. They do not resolve contradictions between sources. They do not reason about what the data means operationally. They show a map with dots on it. The dots do not tell Rajesh which direction to send his boats. A GIS analyst could build custom layers, but Rajesh does not have a GIS analyst at 2 AM during a cyclone. He has a satellite phone and a map taped to a wall.

Option C: Wait for ISRO or NRSC damage assessments. The National Remote Sensing Centre publishes post-disaster damage assessments using satellite imagery. These are high quality. They are also published 24 to 72 hours after the event. For a flood that moves 2 kilometres per hour, a 48-hour-old assessment tells you where the water was, not where it is. Rescue operations happen in the first 12 hours. The assessment arrives after the window has closed.

Option D: Rely on the state SDMA coordination. Each state has a State Disaster Management Authority that theoretically coordinates response. In practice, the SDMA aggregates reports from district collectors, who aggregate reports from block officers, who aggregate reports from village pradhans. Each layer adds latency. Each layer filters

information. By the time a ground-level report reaches the state EOC, it has been compressed, delayed, and stripped of the spatial precision that makes it actionable. The SDMA's "situation report" is published once or twice daily. The disaster does not pause for the reporting cycle.

Option E: Watch the news. Television channels deploy reporters and drones. Their footage is recent and dramatic. It is also unstructured, ungeoreferenced, and selected for visual impact rather than operational utility. A drone shot of a flooded village tells you that village is flooded. It does not tell you how deep the water is, which direction it is flowing, how many people are stranded, what access routes remain open, or whether the bridge 3 kilometres upstream is still passable. The news tells you the story. It does not tell you the situation.

Every existing approach fails in the same way. It either shows data without intelligence, or provides intelligence without timeliness, or offers timeliness without spatial precision. Nobody is simultaneously ingesting river gauge levels, rainfall radar, satellite imagery, social media signals, road network status, and asset positions; resolving contradictions between them; projecting where the situation will be in 6 hours; and generating specific, spatially grounded recommendations that a field commander can act on in the next 15 minutes.

The Contradictions That Matter

Here is where it gets interesting. Consider these real-world scenarios:

Scenario 1: Two sources, two realities.

The CWC gauge at Naraj on the Mahanadi reads 26.3 metres, which is 0.8 metres below danger level. Meanwhile, 47 geotagged tweets from Jajpur district (downstream of Naraj) show knee-deep water on NH-16 and people evacuating by boat. The gauge says "no flood." The social media says "we are drowning." Both are correct. The gauge measures river level at a single point. The flooding in Jajpur is caused by drainage congestion from heavy local rainfall that the river gauge cannot capture. Your system has two data sources that contradict each other. Neither is wrong. What does your system believe? How does it present both truths to Rajesh without creating confusion? Does it fuse them or show them side by side? What weight does it give 47 unverified tweets versus one calibrated instrument?

Scenario 2: The map says the road exists. The road does not exist.

OpenStreetMap shows NH-16 as a clear route from Bhubaneswar to Balasore. Three hours ago, the bridge at Jenapur was submerged. This information exists in a district collector's

WhatsApp message that has not reached anyone outside his group. Rajesh's supply convoy is 45 minutes from Jenapur. If the convoy reaches the bridge and finds it impassable, they lose 3 hours backtracking. Your system needs to ingest not just formal data sources but informal signals: social media reports of road closures, citizen photos of submerged bridges, municipal CCTV where available. How do you validate these signals? A single tweet claiming a bridge is washed out could be true, exaggerated, or completely fabricated. What confidence threshold does your system need before it reroutes a convoy? What is the cost of acting on a false positive versus ignoring a true positive?

Scenario 3: The prediction was wrong, and the boats are in the wrong place.

Six hours ago, IMD's forecast predicted the heaviest rainfall over Kendrapara district. Rajesh deployed three boats there. The rainfall actually concentrated over Jagatsinghpur, 60 kilometres south. Kendrapara is damp. Jagatsinghpur is underwater. The boats are in the wrong district. Redeployment takes 4 hours by road (assuming the roads are passable). Your system showed a prediction. The prediction was wrong. Now what? Does it alert Rajesh that the ground truth has diverged from the forecast? Does it automatically recommend redeployment? Does it estimate the time cost of redeployment versus the risk of waiting? And critically: the next time your system shows a prediction, how does Rajesh calibrate his trust in it, knowing the last prediction was wrong? How does your system handle its own credibility over time?

Scenario 4: Asset tracking is the easy part. Asset allocation is the hard part.

Rajesh knows where his 30 boats are. That is the easy problem. The hard problem is: given the current flood extent, the projected flood progression over the next 6 hours, the population density in affected areas, the road network status, and the redeployment time for each asset, where should the boats be? This is a constrained optimization problem that changes every 30 minutes as the flood evolves. An experienced commander solves it through intuition built over decades. A new commander does not have that intuition. Your system needs to provide resource allocation recommendations that account for spatial constraints (you cannot drive a boat across dry land), temporal constraints (redeployment takes time), population weighting (a flooded village of 5,000 people is higher priority than a flooded field), and uncertainty (the flood front might shift). How do you present an optimization recommendation to someone who does not think in optimization terms? What does "send 2 boats from Kendrapara to Jagatsinghpur via the Devi River route, ETA 2.5 hours" look like on a map that a battalion commander can act on at 3 AM?

Scenario 5: The satellite sees everything, 12 hours late.

Sentinel-2 provides high-resolution imagery that can clearly show flood extent. The revisit time is 5 days. ISRO's Resourcesat can be tasked for emergency acquisition, but processing and delivery take 12 to 24 hours. By the time the satellite image arrives, the flood has advanced, receded, or shifted. The image is the most accurate spatial data your system will ever get, and it is always a picture of the past. Your system needs to use satellite imagery as ground truth for calibrating its other models, not as a real-time feed. How do you reconcile a 12-hour-old satellite image that shows precise flood boundaries with real-time river gauge data that suggests the boundaries have shifted by 8 kilometres? Do you age-discount the satellite data? Do you use it to validate or invalidate the gauge-based projections? What is the half-life of a satellite observation during a fast-moving flood?

Scenario 6: The quiet district is the dangerous one.

Puri district has filed 14 situation reports, 200 tweets are geotagged there, and the district collector is calling every hour. Ganjam district has filed zero reports, has 3 tweets, and the collector is not answering his phone. Which district is in worse shape? The instinct is to focus on Puri because that is where the data is. But the silence from Ganjam might mean the communication infrastructure has collapsed, which would mean Ganjam is in worse shape and nobody knows. Your system has abundant data from some areas and no data from others. The absence of data is itself a signal. How does your system distinguish between “nothing is happening in Ganjam” and “everything is happening in Ganjam but nobody can report it”? Can you use proxy signals (power outage data, cell tower status, upstream river gauge readings) to infer the state of a district that has gone silent?

Scenario 7: Yesterday's rescue is today's hazard.

Rajesh's team rescued 340 people from Erasama block yesterday and moved them to a relief camp at the local school. Today, the flood has expanded. The school is now in the flood zone. The 340 people Rajesh rescued yesterday need to be rescued again. Every rescue operation creates a new dependency. Every relief camp is a potential future rescue site. Your system needs to model not just where the flood is, but where it will be, and whether the locations where people have been moved will remain safe. This is not a point-in-time analysis. It is a trajectory problem. How does your system incorporate flood progression models? How does it flag relief camps that are at risk of being inundated within the next 6, 12, or 24 hours? What does the alert look like when the safe zone becomes the danger zone?

Each of these scenarios demands a different kind of reasoning. Some require multi-source data fusion with conflict resolution. Some require spatial reasoning and pathfinding. Some require temporal projection and uncertainty modelling. Some require interpreting the absence of data as information. Some require constrained optimization under changing

conditions. A system that handles only one of these is a visualization tool. A system that handles all of them is an intelligence platform.

Your Job

Solve Rajesh's problem.

That is the entire brief. Everything below is here to help you think, not to tell you what to build.

Questions to Sit With Before You Design Anything

Do not skim these. Each question points at a design decision that will shape your entire solution. If you cannot answer a question clearly, you are not ready to build that part of the system.

On the nature of the data

A CWC river gauge reading arrives as a JSON payload (or a scraped HTML table) with a station ID, a water level in metres, a timestamp, and a danger level threshold. An IMD rainfall radar image arrives as a GeoTIFF or a bulletin in semi-structured text. A satellite image arrives as a multi-band raster with coordinates and a capture timestamp. A tweet arrives as text with GPS coordinates of uncertain accuracy and zero verification. A district collector's report arrives as a PDF or a WhatsApp voice note. An ADS-B feed gives you helicopter positions in real time as structured JSON. What does this tell you about your ingestion layer? Can a single pipeline handle all of them? What happens when the CWC portal goes down during exactly the kind of event when you need it most?

Disaster data is not just multi-format. It is temporally misaligned. The gauge reading is from 15 minutes ago. The satellite image is from 14 hours ago. The tweet is from 3 minutes ago. The IMD forecast is for the next 24 hours. The district report is from this morning. Your system needs to present all of these as a single coherent picture. But they are pictures of different moments. How do you time-align data sources with fundamentally different cadences? How do you communicate to Rajesh which parts of his operational picture are fresh and which are stale? A 15-minute-old gauge reading and a 14-hour-old satellite image are both "current" in some sense, but their decision value is entirely different.

On the knowledge your system needs but does not have in the input

Your system needs to know things like: the danger level of the Mahanadi at Naraj is 25.5 metres. The carrying capacity of a standard NDRF inflatable boat is 12 persons. The average

travel time from Cuttack to Jagatsinghpur by road is 2 hours in normal conditions and 5 hours during heavy rain. The population of Erasama block is 2,80,000. The school at Erasama that serves as a relief camp has an elevation of 8 metres above mean sea level. NH-16 between Bhubaneswar and Balasore has 4 bridges, each with a different flood tolerance. This knowledge comes from hydrological databases, NDRF operational manuals, census data, infrastructure surveys, and elevation models. Where does each come from? How does your system access it at the right moment? What happens when the population data is from the 2011 census and the actual population has shifted? What happens when a new bridge was built that your system does not know about?

When your system recommends redeploying boats from Kendrapara to Jagatsinghpur, it is making a recommendation with life-or-death consequences. If the recommendation is wrong (the road is impassable, the boats arrive too late, Kendrapara floods after the boats leave), people die. How do you ensure your system is reasoning from actual data and not generating plausible-sounding spatial inferences? How do you distinguish between “this route is viable because I have road status data” and “this route looks viable on the map but I have no current information about its condition”?

On trust and confidence

If your system has gauge data from 8 of 15 monitoring stations in the Mahanadi basin and the 8 stations all show rising levels, what should it say about the 7 stations it cannot reach? The temptation is to interpolate: “If all reporting stations are rising, the unreported ones probably are too.” But rivers are not uniform. A tributary might be falling while the main channel rises. A dam upstream might be releasing water that affects some stations and not others. How does your system represent spatial uncertainty without overwhelming the commander with caveats?

A single unverified tweet claiming a dam has breached versus 15 CWC gauge readings showing normal levels downstream of that dam. What does your system do? Ignore the tweet? Flag it? Alert the commander? Somewhere on the spectrum between “ignore all unverified social media” and “treat every tweet as ground truth” is the right operating point. Where is it? How does your system calibrate it? And what happens when the one tweet it ignored turned out to be the first report of an actual breach?

When the IMD forecast says Category 4 cyclone with 180 kmph winds and the actual landfall is Category 3 with 140 kmph winds, the response mobilization was calibrated for a different disaster. Over-preparation wastes resources. Under-preparation costs lives. Your system inherits the uncertainty of every source it ingests. How do you propagate that uncertainty through your recommendations without paralysing the decision-maker?

On the multi-stakeholder problem

Rajesh is one user. But the same intelligence serves different stakeholders with different needs. The NDRF commander needs tactical deployment recommendations. The state SDMA chief needs a strategic overview across all districts. The district collector needs hyperlocal information about his specific jurisdiction. The relief coordinator needs to know which camps are at capacity and which roads are open for supply trucks. The media coordinator needs damage estimates and rescue statistics. Each stakeholder needs a different view of the same underlying data. Does your system generate one output or five? What is the canonical data model that supports all of these views? How do you prevent information overload for the tactical user while providing sufficient detail for the strategic user?

The NDRF operates alongside the Indian Army, Navy, Air Force, state disaster response forces, district administration, and NGOs. Each organization has its own communication channels, its own coordination protocols, and its own terminology. Your system might show that IAF helicopters are available for aerial rescue, but the NDRF commander cannot task IAF assets directly. He needs to request them through the state EOC. Does your system model the command hierarchy? Should it? What happens when it recommends an action that requires inter-agency coordination that does not exist in practice?

On time pressure and output design

Rajesh is not a data analyst. He is a field commander operating on 3 hours of sleep during a Category 4 cyclone. He needs three things: Where is the situation worst right now? Where will it be worst in 6 hours? Where should I send my teams? Your system's output is not a dashboard. It is a decision support tool operating under extreme time pressure. It needs to be glanceable, spatially grounded, and actionable. What does that look like on a phone screen at 3 AM in driving rain?

If your system generates a recommendation to redeploy three boats from Kendrapara to Jagatsinghpur, what supporting information does Rajesh need to trust and act on that recommendation? The route? The estimated travel time? The risk assessment for both districts? The population at risk in each? The confidence level of the flood projection? Too much information and he cannot parse it. Too little and he cannot justify the decision to his superiors. Where is the right level for someone making life-or-death calls every 30 minutes?

If your system's flood projection shows that a relief camp will be inundated within 4 hours, but the projection is based on a model with a 30% error margin, should it trigger an evacuation alert? Moving 340 people from a camp that might not flood wastes resources and creates panic. Not moving them from a camp that does flood is catastrophic. How does your

system calibrate the action threshold when the cost of false positives and false negatives are both severe but asymmetric?

On structure and resilience

If tomorrow the CWC launches a new API that provides gauge data in a different format, how much of your system would you need to rewrite? If ISRO's Bhuvan adds a real-time flood extent layer, can your system ingest it without architectural changes? If a new social media platform becomes the primary channel for citizen flood reports, can your system adapt? What does your answer tell you about how you should design your data ingestion layer?

When a cyclone makes landfall, cell towers go down, internet connectivity becomes intermittent, and power outages disable data centres. Your system needs to operate in degraded mode. If the CWC feed is unavailable, does your system fail entirely, or degrade to social-media-only situational awareness? If satellite imagery is cloud-covered and useless, what falls back to the primary spatial layer? If Rajesh's internet connection drops to 2G, what is the minimum viable output your system can deliver? Design for the conditions of use, not the conditions of development.

Your system needs to learn from every disaster it monitors. The gap between IMD's forecast and actual rainfall. The correlation between upstream gauge readings and downstream flooding 6 hours later. The typical time between a social media report and official confirmation. The historical accuracy of your own flood projections. How do you design the system so that its performance during the fifth cyclone is measurably better than during the first? What data do you retain? What feedback loops do you build? What privacy and data sovereignty constraints apply when ingesting citizen-generated content?

On the person reading the output

Rajesh is exhausted, under pressure, and responsible for hundreds of lives. He has been coordinating rescues for 36 straight hours. He does not understand GIS terminology. He does not read satellite imagery. He does not parse probability distributions. He needs to know: Where are people drowning right now? Are my teams in the right places? What should I change? If your system gives him a multi-layer geospatial dashboard with 12 toggleable data feeds and a statistical confidence panel, he cannot use it. If your system gives him a single red dot on a map with no context, he cannot act on it. Where is the right level?

When your system surfaces a critical alert (bridge submerged on the supply route, relief camp at risk of inundation, population cluster with no rescue assets nearby), how do you communicate urgency to someone who is already in a state of sustained emergency? Every

alert competes with every other alert. If everything is urgent, nothing is. How does your system triage its own outputs? How does it decide which of five simultaneous alerts reaches the commander's phone screen first? The prioritization logic is not a UX problem. It is a decision architecture problem that determines whether the most important information reaches the right person at the right moment.

If your system recommends a specific action ("redeploy 2 boats via Route X, ETA 2.5 hours") and Rajesh follows it, and the recommendation turns out to be wrong (Route X was flooded, the boats arrived 5 hours late, people died waiting), what is your system's responsibility? How do you design recommendations that preserve the commander's agency rather than replacing his judgment? What is the difference between "Route X appears viable based on gauge data from 2 hours ago" and "Take Route X"? Each framing leads to a different action and a different accountability structure. What is the design of honesty that supports decision-making under uncertainty without creating false precision?